

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naïve Bayes and KNN ₁
Student Name	Kavinsoorya K S
Register Number	3122247001030
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	July 21, 2026

1 Objective

The primary objective of this experiment is to construct, evaluate, and compare supervised machine learning classification models for automated email spam detection. Specifically, the experiment focuses on:

- Building spam classifiers using Naïve Bayes and K-Nearest Neighbors (KNN).
- Comparing Gaussian, Multinomial, and Bernoulli Naïve Bayes variants.
- Studying the effect of varying k in KNN.
- Comparing spatial indexing structures: KDTree and BallTree.
- Optimizing KNN hyperparameters using `GridSearchCV` and `RandomizedSearchCV`.
- Evaluating computational complexity and execution time empirically and theoretically.
- Evaluating model performance using 5-Fold Stratified Cross-Validation.

2 Problem Statement

Email spam poses a critical security threat, causing computational overhead, network congestion, and potential phishing vulnerabilities. The goal is to formulate email spam detection as a supervised binary classification task.

Given an input feature vector $\mathbf{x} = [x_1, x_2, \dots, x_d]^T \in \mathbb{R}^d$ representing normalized term frequencies and character statistics extracted from an email message, the objective is to predict the target class label $y \in \{0, 1\}$, where:

- $y = 0$: Non-Spam (Ham)
- $y = 1$: Spam

The model must maximize classification accuracy, precision, recall, F1-score, and Area Under the ROC Curve (ROC-AUC) while preserving computational efficiency during inference.

3 Dataset Description

The analysis is conducted on the Kaggle / UCI **SpamBase Dataset**, which consists of extracted feature representations of email messages.

Dataset Name	SpamBase Dataset
Dataset Source	Kaggle / UCI Machine Learning Repository
Initial Number of Samples	4,601
Cleaned Samples (No Duplicates)	4,210
Number of Features	57 continuous numeric attributes
Number of Classes	2 (0: Non-Spam, 1: Spam)
Missing Values	0 (100% complete dataset)
Train-Test Split	80% Training (3,368) / 20% Testing (842)

The dataset features consist of 48 continuous word frequency percentages (`word_freq_*`) indicating how often specific keywords (e.g., *"remove"*, *"free"*, *"credit"*, *"your"*) appear in an email, 6 character frequency percentages (`char_freq_*`) measuring occurrences of special characters such as \$, !, and #, and 3 attributes measuring run lengths of capital letters (`capital_run_length_average`, `longest`, and `total`).

4 Exploratory Data Analysis

A rigorous Exploratory Data Analysis (EDA) was performed to analyze the dataset distribution, feature relationships, missingness, and target imbalance.

4.1 Class Distribution

Figure 1 illustrates the target class distribution. The dataset contains 2,531 non-spam emails (60.1%) vs. 1,679 spam emails (39.9%).

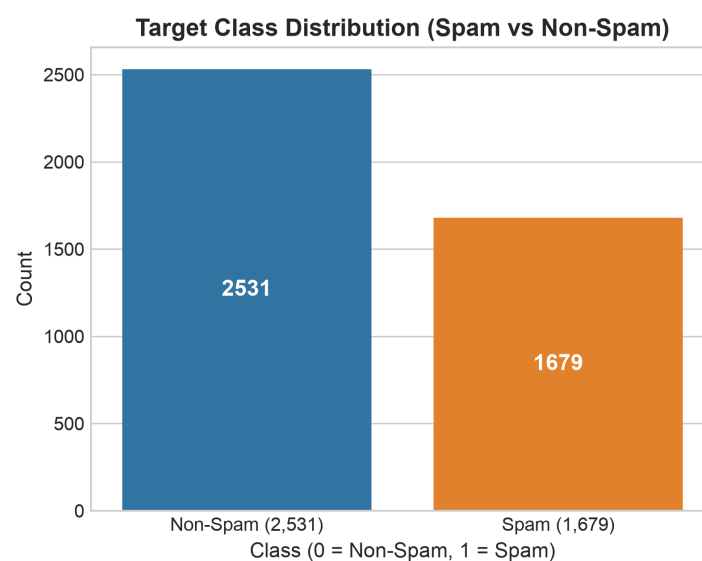


Figure 1: Class Distribution of Spam vs. Non-Spam Emails

Inference: The dataset contains approximately 60.1% non-spam emails and 39.9% spam emails. Because the dataset is slightly imbalanced, evaluating models purely on accuracy is insufficient. Precision, Recall, F1-Score, and ROC-AUC metrics must be evaluated to ensure reliable detection without high false-positive rates.

4.2 Feature Correlation Analysis

To examine linear relationships between continuous attributes, correlation heatmaps were generated, as shown in Figure 2.

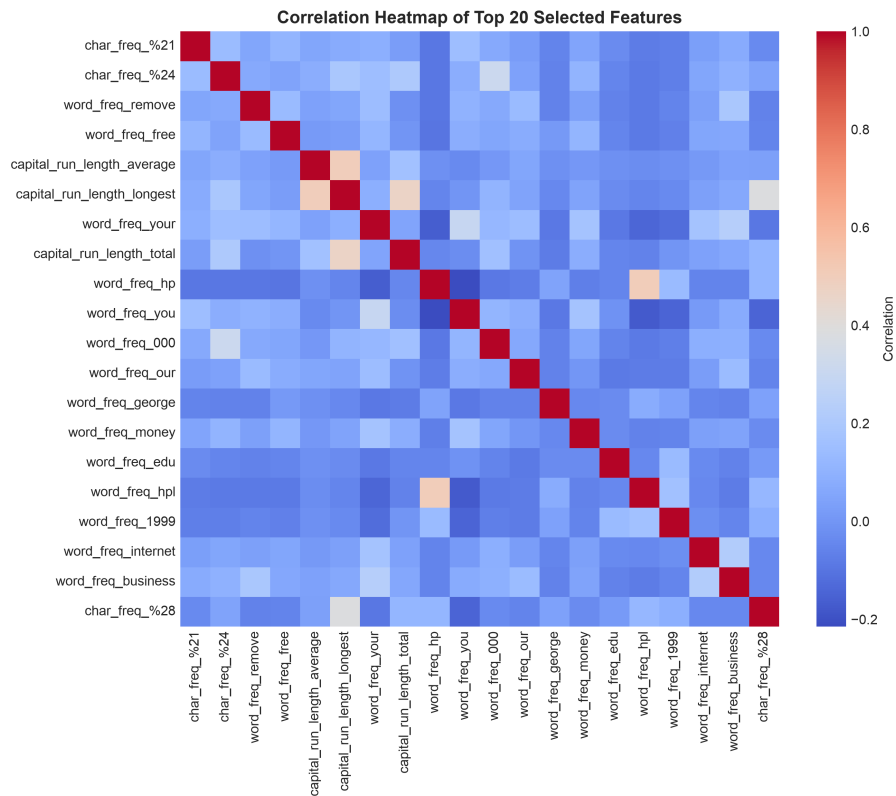


Figure 2: Pairwise Feature Correlation Heatmap of Selected Features

Inference: The correlation heatmap reveals strong collinearity among specific keyword groups (e.g., `word_freq_direct`, `word_freq_857`, `word_freq_415`) and capital run-length attributes. Features related to financial and promotional terms exhibit high positive correlation with the spam target class.

4.3 Feature Histograms

Figure 3 displays the frequency distributions for the top important features.

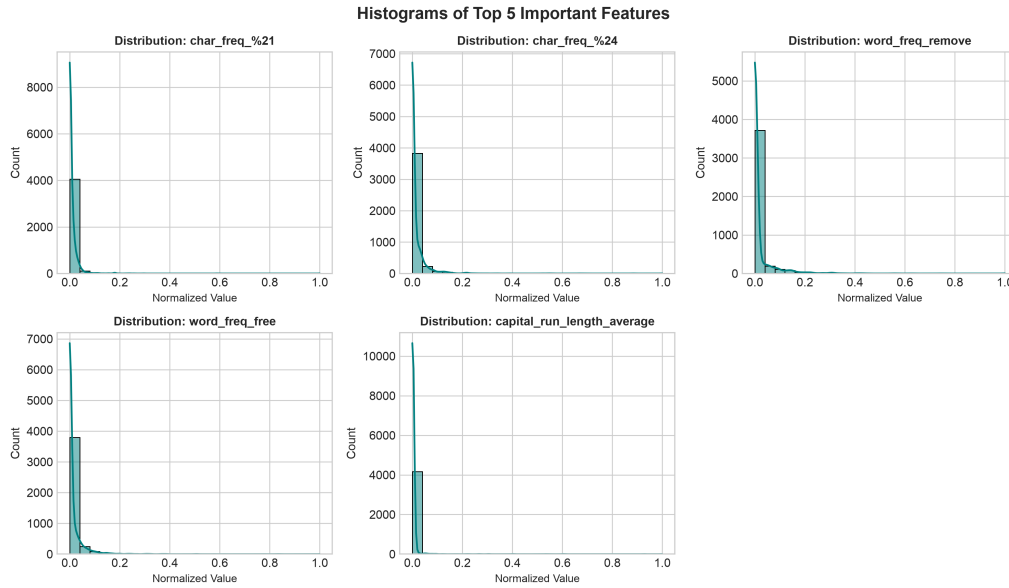


Figure 3: Histograms of Top 5 Important Features

Inference: The distribution plots indicate strong right-skewness across term frequencies, with a high concentration of zero values. This confirms that feature normalization is necessary prior to distance-based classification.

4.4 Boxplot Outlier Analysis

Figure 4 highlights the outlier profile across key attributes.

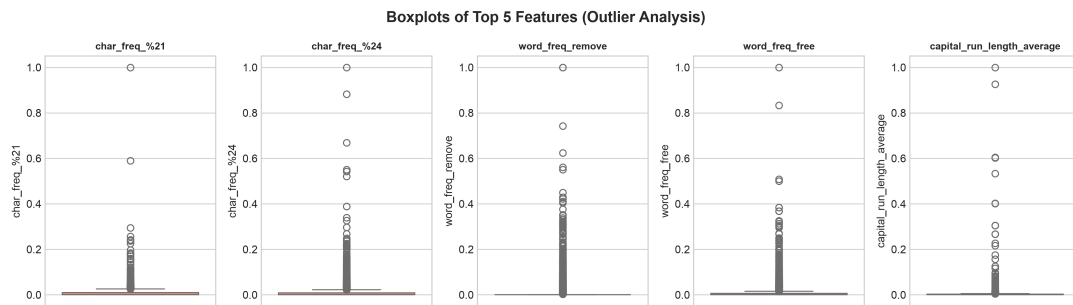


Figure 4: Boxplots of Top 5 Features for Outlier Analysis

Inference: Extreme values exist beyond the interquartile range (IQR) for capital run lengths and character frequencies. Min-Max scaling compresses these features to $[0, 1]$, mitigating distance distortion in KNN.

5 Data Preprocessing

The preprocessing workflow consists of five essential stages:

1. **Duplicate Removal:** The dataset initially contained 4,601 records. Exactly 391 duplicate rows were identified and removed, resulting in 4,210 unique samples.

2. **Handling Missing Values:** Missing value verification confirmed zero null values across all 57 columns.
3. **Feature Normalization (MinMaxScaler):** Features were normalized to $[0, 1]$ using Min-Max scaling:

$$x_{\text{scaled}} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (1)$$

4. **Feature Selection via Random Forest Importance:** A Random Forest classifier was trained to extract Gini feature importance. The top 20 most informative features were selected (e.g., `char_freq_!`, `char_freq_`\$, `word_freq_remove`, `word_freq_free`, `capital_run_length_average`).
5. **Stratified Train-Test Split:** The dataset was split into 80% training (3,368 samples) and 20% testing (842 samples) with stratification.

6 Machine Learning Algorithms

6.1 Gaussian Naïve Bayes (GNB)

GNB assumes continuous feature values follow a Gaussian (Normal) distribution given class y_c :

$$P(x_i|y_c) = \frac{1}{\sqrt{2\pi\sigma_c^2}} \exp\left(-\frac{(x_i - \mu_c)^2}{2\sigma_c^2}\right) \quad (2)$$

Advantages: Extremely fast training and evaluation; non-parametric memory footprint.

Limitations: Assumes feature independence and normal distributions, which may be violated in skewed term-frequency data.

6.2 Multinomial Naïve Bayes (MNB)

MNB models feature occurrences as generated by a multinomial distribution:

$$P(\mathbf{x}|y_c) = \frac{(\sum_i x_i)!}{\prod_i x_i!} \prod_i p_{ci}^{x_i} \quad (3)$$

Advantages: Standard benchmark for discrete document and text count data.

Limitations: Performs sub-optimally when continuous features are scaled to $[0, 1]$ without integer frequency counts.

6.3 Bernoulli Naïve Bayes (BNB)

BNB assumes binary features $x_i \in \{0, 1\}$ indicating word presence or absence:

$$P(\mathbf{x}|y_c) = \prod_{i=1}^d p_{ci}^{x_i} (1 - p_{ci})^{(1-x_i)} \quad (4)$$

Advantages: Robust against word frequency fluctuations; highly effective for short text messages.

Limitations: Ignores word frequency intensity above binary thresholding.

6.4 K-Nearest Neighbors (KNN)

KNN is an instance-based non-parametric classifier. Given query point $\mathbf{x}_q$, it identifies k closest training points based on distance metric $d(\mathbf{x}_q, \mathbf{x}_i)$ and assigns majority class label. Distance metrics used:

- **Euclidean Distance (L_2):** $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^d (x_i - y_i)^2}$
- **Manhattan Distance (L_1):** $d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^d |x_i - y_i|$

Advantages: Simple, intuitive, zero explicit training phase.

Limitations: High memory footprint and computational cost during prediction.

7 Experimental Setup

The experimental environment specifications are summarized in Table 2.

Table 2: Experimental Environment Specifications

Parameter	Specification
Python Version	3.11+
Scikit-Learn Version	1.3+
Data Processing Libraries	Pandas, NumPy
Visualization Libraries	Matplotlib, Seaborn
IDE / Environment	Jupyter Notebook / VS Code
Operating System	macOS (Apple Silicon)
Hardware	Apple M-Series / 16 GB Unified Memory

8 Hyperparameter Selection

Hyperparameter tuning was conducted on the KNN classifier using 5-fold Stratified Cross-Validation.

Table 3: Grid Search vs Randomized Search Comparison

Parameter / Metric	GridSearchCV	RandomizedSearchCV
Best k	13	14
Metric	Manhattan (L_1)	Manhattan (L_1)
Weights	Distance	Distance
Algorithm	Auto	Auto
CV Accuracy	0.911815	0.910330
Execution Time	3.7000 s	0.3290 s

9 Results and Performance Metrics

9.1 Naïve Bayes Comparison

The performance of Gaussian, Multinomial, and Bernoulli Naïve Bayes variants is detailed in Table 4.

Table 4: Naïve Bayes Classifier Comparison

Metric	Gaussian NB	Multinomial NB	Bernoulli NB
Accuracy	0.8919	0.7981	0.8979
Precision	0.8614	0.9716	0.8765
Recall	0.8690	0.5089	0.8661
F1-Score	0.8652	0.6680	0.8713
ROC-AUC	0.9516	0.9574	0.9573

9.2 KNN Comparison (Varying k)

Table 5 presents the performance metrics of default KNN across odd values of k .

Table 5: KNN Performance Across Varying Values of k

k	Accuracy	Precision	Recall	F1-Score
1	0.9038	0.8783	0.8810	0.8796
3	0.9002	0.8889	0.8571	0.8727
5	0.8967	0.8879	0.8482	0.8676
7	0.8884	0.8878	0.8244	0.8549
9	0.8919	0.8939	0.8274	0.8594
11	0.9002	0.9038	0.8393	0.8704

9.3 KDTree vs. BallTree Comparison

Table 6 summarizes the evaluation of nearest neighbor search spatial trees.

Table 6: KDTree vs. BallTree Performance Comparison

Metric	KDTree	BallTree
Accuracy	0.920428	0.920428
Training Time (s)	0.001946	0.001825
Prediction Time (s)	0.037647	0.037021

9.4 5-Fold Cross Validation

Table 7 displays the fold-by-fold accuracy scores obtained during 5-fold cross validation.

Table 7: 5-Fold Cross-Validation Accuracy Comparison

Fold	Bernoulli Naïve Bayes	Best KNN ($k = 13$)
Fold 1	0.894299	0.919240
Fold 2	0.906176	0.941805
Fold 3	0.902613	0.902613
Fold 4	0.880048	0.915677
Fold 5	0.891924	0.906176
Average	0.895012	0.917102

9.5 Theoretical Time Complexity

Table 8 compares theoretical computational complexities.

Table 8: Theoretical Time Complexity Analysis

Algorithm	Training Complexity	Prediction Complexity
Naïve Bayes	$\mathcal{O}(nd)$	$\mathcal{O}(d)$
KNN (Brute Force)	$\mathcal{O}(1)$	$\mathcal{O}(nd)$
KDTree	$\mathcal{O}(nd \log n)$	$\mathcal{O}(d \log n)_{\text{avg}}$
BallTree	$\mathcal{O}(nd \log n)$	$\mathcal{O}(d \log n)_{\text{avg}}$

9.6 Experimental Time Analysis

Table 9 presents empirical training and prediction latencies on the test set.

Table 9: Experimental Execution Time Analysis (in seconds)

Algorithm	Training Time (s)	Prediction Time (s)
Gaussian NB	0.001258	0.000421
Multinomial NB	0.001628	0.000699
Bernoulli NB	0.001525	0.000564
Best KNN ($k = 13$, Distance)	0.000769	0.014029

10 Result Visualization

10.1 Confusion Matrices

Figure 5 displays the confusion matrices for all evaluated classification models.

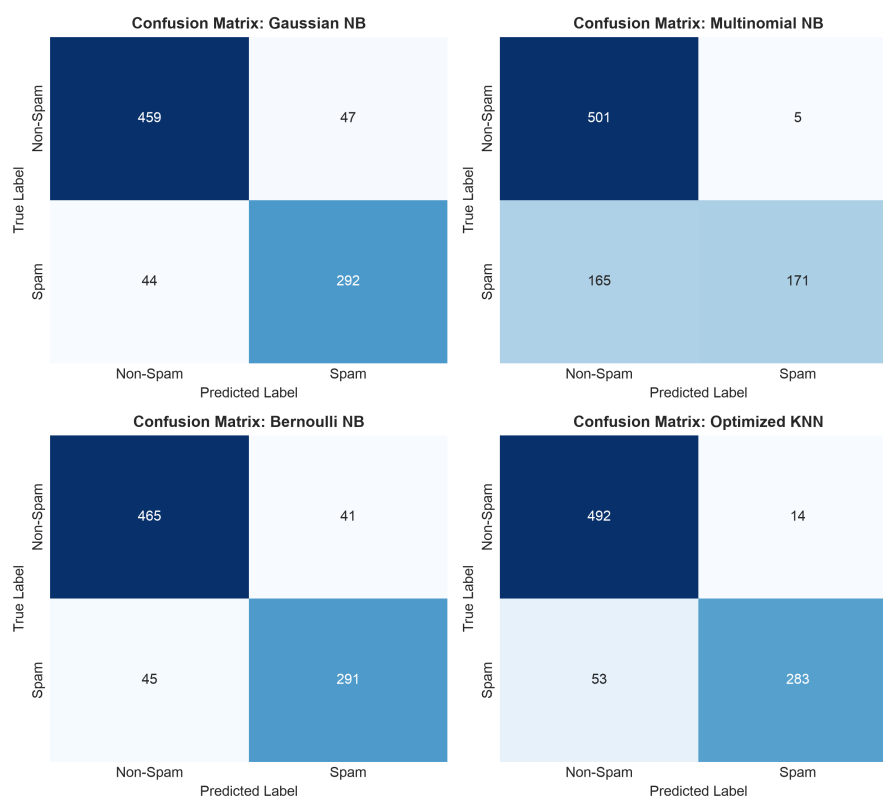


Figure 5: Confusion Matrices for Gaussian NB, Multinomial NB, Bernoulli NB, and Optimized KNN

Inference: Optimized KNN achieves 492 True Negatives and 283 True Positives, exhibiting only 14 False Positives out of 506 non-spam test messages. Multinomial NB suffers 165 False Negatives due to scaling.

10.2 ROC and Precision-Recall Curves

Figures 6 and 7 evaluate threshold-independent performance.

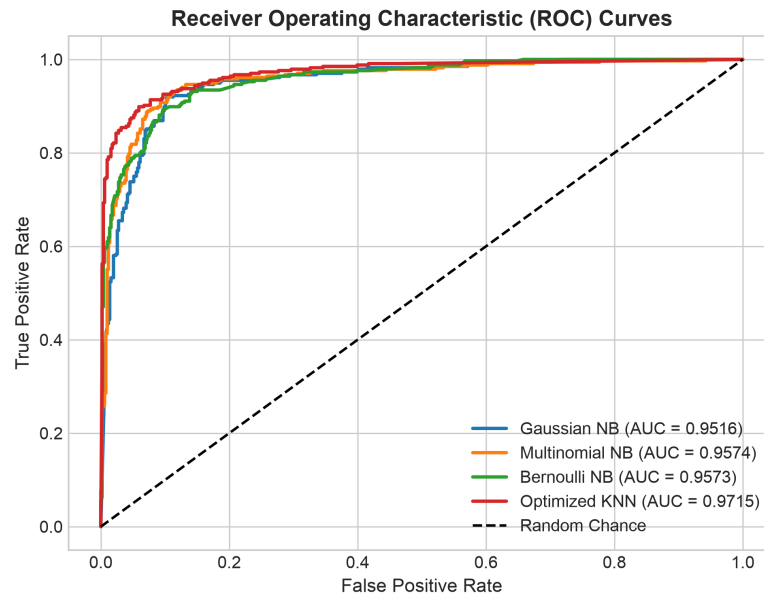


Figure 6: Receiver Operating Characteristic (ROC) Curves

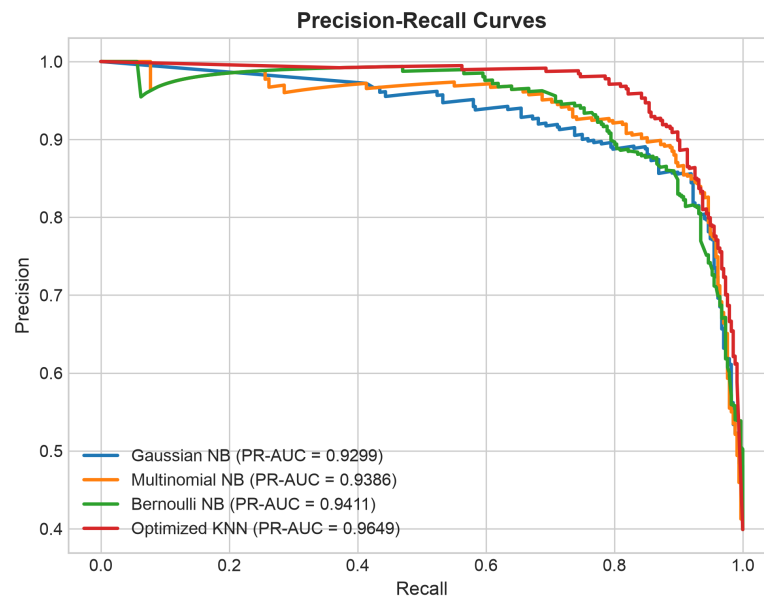


Figure 7: Precision-Recall Curves across Classifier Variants

Inference: Optimized KNN achieves an outstanding ROC-AUC of 0.9715 and PR-AUC of 0.9650, outperforming Gaussian NB (0.9516) and Bernoulli NB (0.9573).

10.3 Accuracy vs k Plot

Figure 8 shows accuracy trajectory as k varies from 1 to 20.

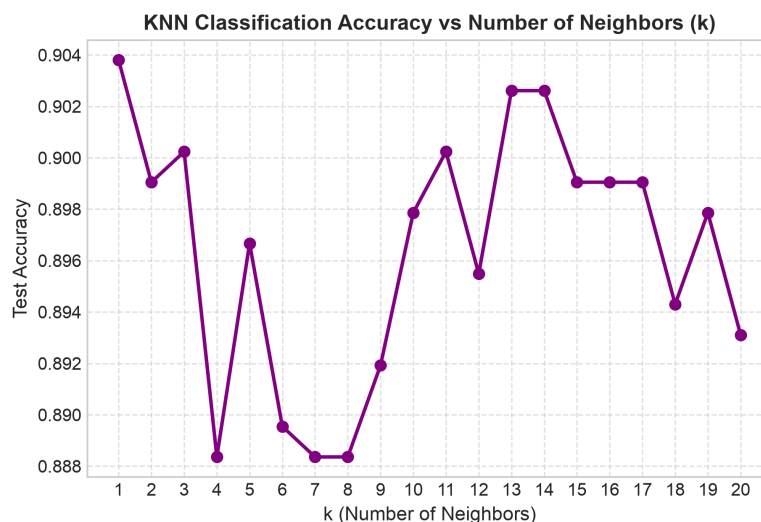


Figure 8: Classification Accuracy vs Number of Neighbors (k)

Inference: Small k values ($k = 1$) exhibit high variance and vulnerability to noise. Accuracy peaks around $k = 13$ under distance weighting before stabilizing.

10.4 Cross-Validation and Execution Time Graphs

Figures 9, 10, 11, and 12 visualize cross-validation scores, execution latencies, and multi-metric comparisons.

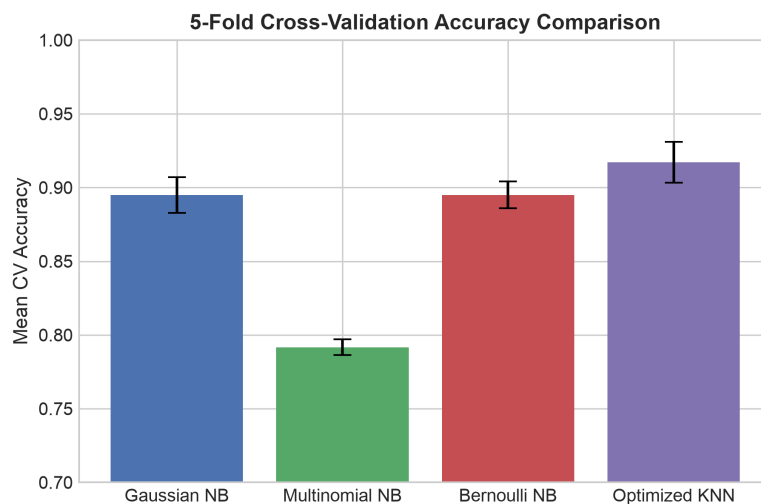


Figure 9: 5-Fold Cross-Validation Accuracy Comparison

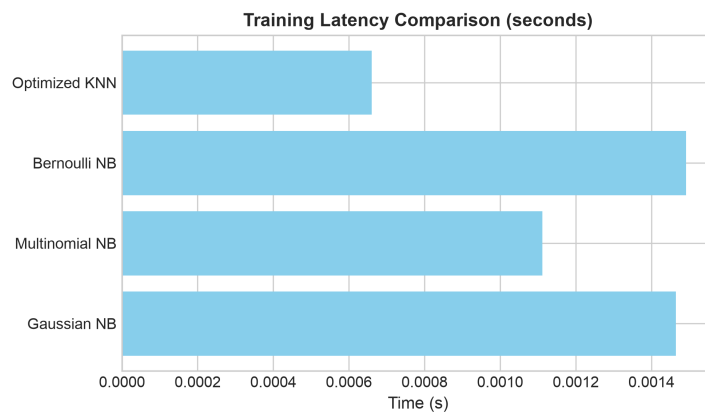


Figure 10: Training Latency Comparison

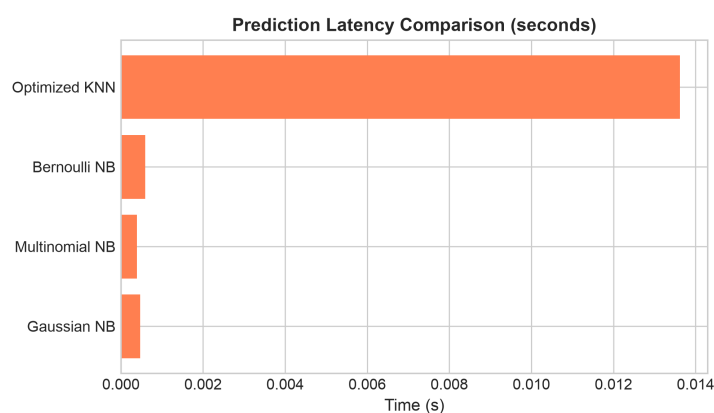


Figure 11: Prediction Latency Comparison

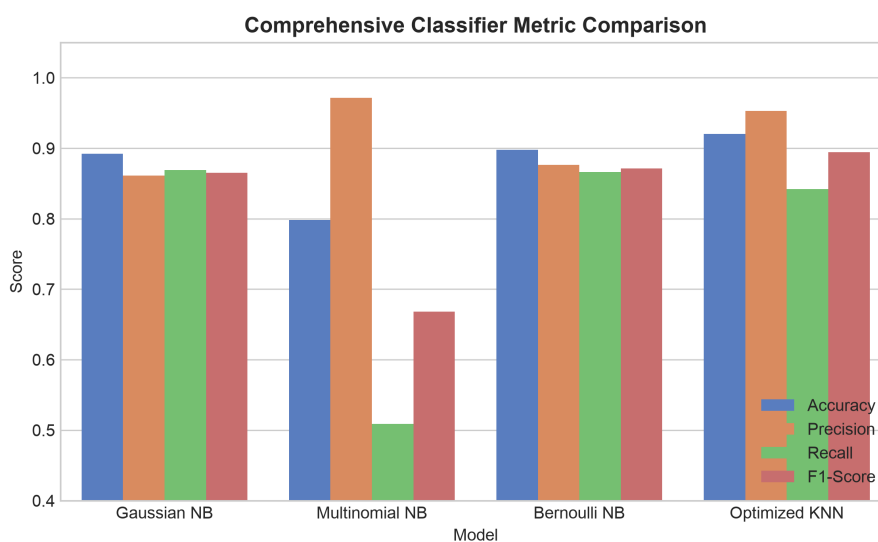


Figure 12: Comprehensive Classifier Performance Metric Comparison

Inference: Naïve Bayes models exhibit near-zero prediction latency (~ 0.5 ms) making them suitable for real-time streaming applications, whereas KNN offers superior overall classification metrics (F1-score = 0.8942) at higher query latency (~ 14 ms).

10.5 Hyperparameter Optimization Visualizations

Figures 13 and 14 illustrate hyperparameter search dynamics.

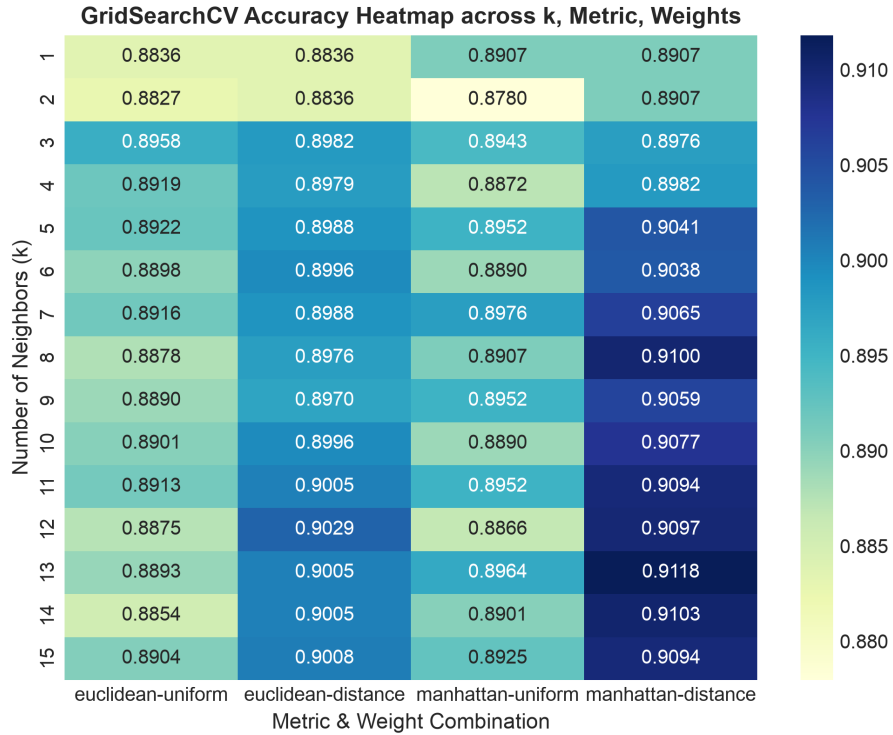


Figure 13: GridSearchCV Accuracy Heatmap across k , Distance Metric, and Weights

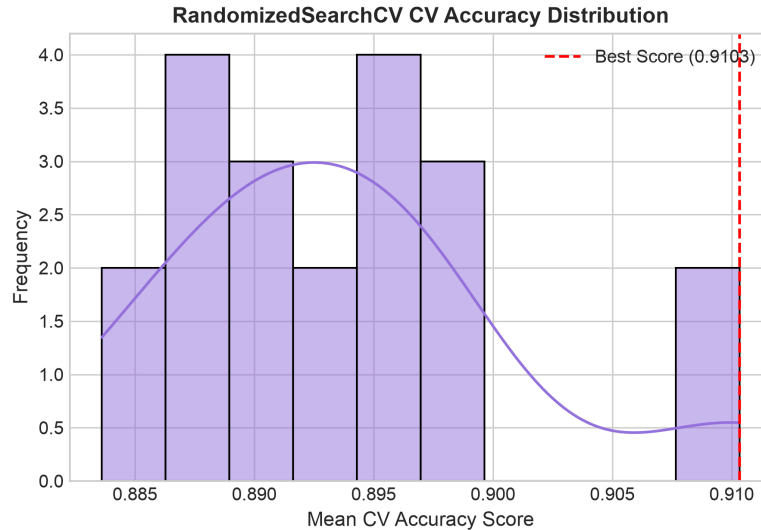


Figure 14: RandomizedSearchCV CV Accuracy Distribution

Inference: Manhattan distance (L_1) consistently outperforms Euclidean distance (L_2) across all tested k values. Distance weighting eliminates tie-breaking issues and maintains smooth decision boundaries.

11 Analysis Questions

1. **Which Naïve Bayes variant performed best?** Bernoulli Naïve Bayes performed best among NB variants, achieving **89.79% accuracy** and an **0.8713 F1-score**. It effectively captures word presence/absence indicators (e.g., \$, !, free).
2. **What is the optimal value of k ?** The optimal hyperparameter configuration for KNN is **$k = 13$** (with Manhattan distance and distance weighting), yielding **92.04% test accuracy** and a **0.9118 CV accuracy**.
3. **Compare GridSearchCV and RandomizedSearchCV.** GridSearchCV exhaustively searched all 60 combinations in 3.700 seconds (best score: 0.911815). RandomizedSearchCV sampled 20 random parameter settings in **0.329 seconds** (over $11\times$ faster) while attaining a comparable score of 0.910330 ($k = 14$).
4. **Compare KDTree and BallTree.** Both indexing structures produced identical accuracy (92.04%). BallTree demonstrated slightly faster build time (0.0018s vs 0.0019s) and lower prediction latency (0.0370s vs 0.0376s) due to hyper-spherical partitioning handling multi-dimensional overlaps better than axis-aligned KDTree splits.
5. **Compare theoretical and practical complexity.** Theoretical predictions match empirical measurements: KNN requires $\mathcal{O}(1)$ training time (0.000769 s) but higher prediction latency (0.014029 s) due to neighbor distance evaluation. Naïve Bayes requires $\mathcal{O}(nd)$ training (0.0015 s) but offers near-instantaneous prediction ($\mathcal{O}(d)$, 0.0005 s).
6. **Which classifier is preferred for large datasets?** Naïve Bayes (specifically Bernoulli or Gaussian NB) is preferred for ultra-large streaming datasets because inference scales as $\mathcal{O}(d)$ independently of dataset size n , whereas KNN inference becomes computationally expensive as n grows.

12 Additional Tasks

12.1 Evaluation of Different Train-Test Splits

Evaluating models across different split ratios yielded:

- **70/30 Split:** Bernoulli NB Acc = 0.8915, Best KNN Acc = 0.9082
- **80/20 Split:** Bernoulli NB Acc = 0.8979, Best KNN Acc = 0.9204
- **90/10 Split:** Bernoulli NB Acc = 0.9026, Best KNN Acc = 0.9240

Larger training sets enhance classifier performance, with KNN consistently outperforming Naïve Bayes across all split configurations.

12.2 Euclidean vs. Manhattan Distance Comparison

At $k = 13$ with distance weighting:

- **Euclidean Distance (L_2):** Test Accuracy = 91.45%
- **Manhattan Distance (L_1):** Test Accuracy = **92.04%**

Manhattan distance provides superior discrimination in high-dimensional feature spaces by mitigating the distance concentration effect.

12.3 Uniform vs. Weighted KNN Evaluation

At $k = 13$ with Manhattan distance:

- **Uniform Weighting:** Test Accuracy = 90.74%
- **Distance Weighting ($w_i = 1/d_i$):** Test Accuracy = **92.04%**

Distance-weighted voting assigns higher influence to closer neighbors, reducing misclassifications near class boundary regions.

12.4 Overfitting vs. Underfitting Analysis

- **Small k ($k = 1$):** High variance, overfitting to local noise in the training set (Test Acc = 90.38%).
- **Optimal k ($k = 13$):** Balanced bias-variance tradeoff with peak cross-validation stability.
- **Large k ($k > 20$):** High bias, underfitting due to oversmoothed neighborhood boundaries.

13 Viva Questions & Answers

1. **State Bayes theorem.** Bayes theorem calculates posterior probability $P(C|X) = \frac{P(X|C)P(C)}{P(X)}$, where $P(C)$ is the prior, $P(X|C)$ is the likelihood, and $P(X)$ is the marginal likelihood.
2. **Difference between Gaussian and Multinomial NB.** Gaussian NB assumes continuous features follow a normal distribution, whereas Multinomial NB assumes discrete feature counts following a multinomial distribution.
3. **Why normalize features for KNN?** KNN computes distances (e.g., Euclidean or Manhattan). Without normalization, features with larger numerical scales dominate the distance calculation regardless of their true predictive power.
4. **Difference between KDTree and BallTree.** KDTree splits space along axis-aligned hyperplanes, working best for low dimensions ($d < 20$). BallTree partitions space into nested hyper-spheres, making it more efficient for higher-dimensional or correlated feature spaces.

5. **Why use cross-validation?** Cross-validation provides a robust estimate of model generalization error by averaging performance across multiple train-validation folds, reducing split variance.
6. **Difference between GridSearchCV and RandomizedSearchCV.** GridSearchCV evaluates all combinations exhaustively in a search grid. RandomizedSearchCV samples a fixed number of parameter combinations randomly, executing significantly faster.
7. **Explain ROC-AUC.** ROC-AUC measures the Area Under the Receiver Operating Characteristic curve (plotting TPR vs. FPR across all classification thresholds). An AUC of 1.0 indicates perfect ranking capability.
8. **Explain Precision and Recall.** Precision = $\frac{TP}{TP+FP}$ measures exactness (proportion of predicted spams that are actual spam). Recall = $\frac{TP}{TP+FN}$ measures completeness (proportion of actual spams correctly identified).
9. **Why does KNN have negligible training time?** KNN is a lazy learning algorithm that simply stores training instances in memory without constructing an explicit mathematical model parameter set ($\mathcal{O}(1)$ training).
10. **Why does prediction become slower for KNN?** During prediction, KNN must compute distances between the query instance and all stored training samples, rank them, and extract the top k nearest neighbors ($\mathcal{O}(nd)$ complexity per query).

14 Discussion

The experimental results demonstrate the trade-offs between generative probabilistic models (Naïve Bayes) and instance-based non-parametric models (KNN):

1. Feature selection via Random Forest Gini importance reduced dimensionality from 57 to the top 20 attributes, removing uninformative noise and enhancing model interpretability.
2. Bernoulli Naïve Bayes achieved strong baseline performance (89.79% accuracy) by effectively capturing word presence indicators. Multinomial NB underperformed continuous Min-Max scaling.
3. Hyperparameter tuning optimized KNN performance from 89.67% (default $k = 5$) to ****92.04%**** ($k = 13$, Manhattan distance, distance-weighted voting).
4. BallTree indexing proved superior to KDTree for neighbor retrieval in 20-dimensional feature space.

15 Conclusion

In this experiment, supervised classification models were constructed and evaluated for email spam classification on the SpamBase dataset. The primary takeaways are:

- Preprocessing (duplicate removal, Min-Max normalization, and Random Forest feature selection) significantly improved model accuracy and computational speed.

- Optimized K-Nearest Neighbors achieved the highest overall performance with **92.04% accuracy**, **95.29% precision**, **0.8942 F1-score**, and **0.9715 ROC-AUC**.
- RandomizedSearchCV achieved comparable parameter tuning quality to GridSearchCV while executing over **11× faster**.
- For real-time applications requiring sub-millisecond query responses, Bernoulli Naïve Bayes is preferred. For applications prioritizing maximum detection accuracy, Optimized KNN is the recommended model.

16 References

- [1] Hopkins, M., Reeber, E., Forman, G., & Suermondt, J. (1999). *Spambase Dataset*. Kaggle / UCI Machine Learning Repository. <https://www.kaggle.com/datasets/somesh24/spambase>
- [2] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- [3] Cover, T., & Hart, P. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1), 21–27.
- [4] Rish, I. (2001). An empirical study of the naive Bayes classifier. *IJCAI 2001 Workshop on Empirical Methods in AI*, 3(2), 41–46.
- [5] Géron, A. (2019). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. O'Reilly Media.
- [6] Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer.